SaaS Starter Kit Guide

Technical Research Report: Architecting a Secure Paywall Around Open Source AI

1. Introduction: The Problem with Open Source AI

The modern artificial intelligence landscape is saturated with thousands of highly capable, genuinely useful open-source repositories. However, a significant majority of these projects will never transition into viable, revenue-generating products[1]. The barrier to entry is rarely the core technology; rather, it is the extensive infrastructure required to monetize it. Historically, transforming a raw repository into a Software-as-a-Service (SaaS) platform necessitated weeks of engineering effort dedicated to building authentication systems, billing integrations, quota enforcement mechanisms, and secure frontend-to-backend handoffs[1].

This technical report outlines a streamlined methodology for bridging this gap, utilizing a specific case study: **MiroFish**. MiroFish is a multi-agent prediction engine hosted on Replit. It allows users to upload documents, generates hundreds of AI agents with distinct personalities, runs them through a dual social simulation, and outputs a structured report for under a dollar per run[1]. The objective of this guide is to demonstrate how a robust SaaS paywall—complete with Google login, a free tier, and Stripe billing—was architected around the existing MiroFish Replit backend in a single evening[1].

2. The Architecture Stack

To achieve rapid deployment without compromising security or scalability, the architecture relies on four foundational pillars. Each tool was selected for its specific interoperability and ability to abstract complex backend logic[1].

> **Industry Context: The Headless SaaS Model**
>
> This architecture exemplifies the "Headless SaaS"; trend, where the frontend presentation layer (Lovable) is decoupled from the core business logic and data layer (Supabase). This separation allows for rapid UI iteration without disrupting backend stability and enables multiple frontends (e.g., web, mobile) to consume the same secure backend APIs.

2.1 Lovable (Frontend & UI Generation)

Lovable serves as the AI app builder for the Vue.js dashboard. Its critical advantage is its native integration with Supabase. Unlike generic code generators that leave developers to manually wire authentication and database connections, Lovable connects directly to a live Supabase project, generating code against a real backend schema[1]. This eliminates the "guesswork" typically associated with connecting frontends to backend services[2].

2.2 Supabase (Identity, State, and Logic)

Supabase acts as the central ";trust anchor" of the entire system[1]. In this distributed architecture, neither the frontend (Lovable) nor the backend (Replit) inherently trusts the other. Instead, both defer to Supabase as the single source of truth for identity and authorization. Building an AI product requires three core backend components: identity management, state persistence, and fast execution logic. Supabase provides all three via:

- **PostgreSQL Database:** Equipped with Row-Level Security (RLS) to ensure data isolation between users[1].
- **Edge Functions:** Running globally on Deno, these functions handle atomic operations like quota gating[1, 2].
- **Authentication:** Lovable Cloud utilizes Supabase under the hood. Consequently, the JSON Web Token (JWT) generated upon user login is a native Supabase JWT, which can be independently verified by external services like Replit[1].

2.3 Stripe (Billing & Subscriptions)

Stripe is utilized to handle all payment processing. By leveraging Stripe Checkout and a single webhook endpoint connected to Supabase, the complexities of subscription management are entirely abstracted[1].

2.4 Replit (Simulation Engine Backend)

Replit hosts the core MiroFish application, written in Python using the Flask framework. It handles multi-agent orchestration and knowledge graph builds via the OpenRouter API. In this architecture, Replit acts as a protected resource server that only executes tasks when provided with a valid, Supabase-signed JWT[1].

3. The Build Process

3.1 Generating the Dashboard with Lovable

The frontend generation process begins with a blank canvas in Lovable. By providing a highly specific, declarative prompt, Lovable can generate the entire UI and wire it to the Supabase backend simultaneously. Below is the exact prompt utilized to generate the MiroFish SaaS dashboard[1]:

```
Build MiroFish, a SaaS dashboard that gates access to an AI
simulation backend at https://<replit-url>.replit.dev.

Auth: Google sign-in via Lovable Cloud.

Plans:
- Free: 3 simulations/month
- Pro: $19/mo, 100 simulations/month (Stripe via Lovable's
  built-in integration)

Pages: landing, login, pricing, and a protected dashboard
showing tier, monthly usage progress bar, a "New run" button
(prompt + optional file URL), and a list of recent runs.

How a run works: user submits a prompt → server checks quota
atomically → logs the run → opens the Replit app in a new tab
with the prompt and the user's auth token in the URL. Replit
verifies the token before doing any work.

Data: profiles, subscriptions, simulation_runs — all with
row-level security so users only see their own data.

Clean modern SaaS design, mobile responsive.
```

This prompt instructs Lovable to generate a sequence of pages (Landing, Login, Dashboard, Pricing) and establishes the core data models. Lovable automatically provisions three tables in the connected Supabase instance: `profiles`, `subscriptions`, and `simulation_runs`, applying Row-Level Security (RLS) policies to all three to guarantee data privacy[1, 2].

3.2 Configuring Supabase Under the Hood

While Lovable handles the initial provisioning, the Supabase backend is doing the heavy lifting. The generated schema includes an Edge Function designed to act as an atomic quota gate (identified as `try_consume_run`)[2]. When a user attempts a run, this function atomically checks their subscription tier against their current usage, logs the run if permitted, and facilitates the handoff to Replit[1, 2]. Furthermore, a Stripe webhook is configured to automatically write subscription status updates (e.g., upgrading to "Pro") directly into the `subscriptions` table[1].

> **Best Practice: Atomic Quota Gates**
>
> To prevent race conditions where multiple concurrent requests might bypass quota limits, the Edge Function utilizes PostgreSQL row locking. By employing `SELECT ... FOR UPDATE` on the user's subscription record, the system ensures that quota checks and decrements happen atomically. This guarantees that even if a user triggers 10 runs simultaneously, the system accurately enforces the limit without overselling.

3.3 Locking Down the Replit Backend

With the frontend and database established, the existing open-source Replit application must be secured. It must transition from a public endpoint to a protected API that only accepts requests from authenticated Lovable users. This was achieved using the following prompt within the Replit environment[1]:

```
Lock down my Flask app at mirofish/backend so only signed-in
users from my Lovable Supabase dashboard can use it.

On the backend, validate a Supabase JWT (from
Authorization: Bearer or ?token=) on every /api/* route,
skip /health, return 401 if invalid. Use SUPABASE_URL +
SUPABASE_ANON_KEY env vars. If those secrets aren't set,
pass through silently. Restrict CORS to *.lovable.app and
*.lovable.dev.

On the Vue frontend at artifacts/mirofish, grab ?token= from
the URL on load, strip it from the address bar, and attach it
as a Bearer header on every Axios call.

My Lovable app will redirect users to
https://<replit-url>/?token=<access_token>. Tell me what
files changed and what secrets to set.
```

This implementation introduces a Flask JWT Middleware that verifies the token on every API route. It requires setting two environment variables (`SUPABASE_URL` and `SUPABASE_ANON_KEY`) to allow Replit to communicate with Supabase for token validation[1].

4. Security Architecture: The JWT Handoff

The most critical component of this distributed architecture is the secure handoff between the Lovable frontend and the Replit backend. As illustrated in the system architecture diagram (Lovable ↔ Supabase ↔ Replit)[1, 2], the security model relies on a dual-checkpoint system where neither the frontend nor the backend inherently trusts the other; both defer to Supabase as the ultimate source of truth[1].

The Handoff Lifecycle & Best Practices

1. **Authentication:** The user logs into the Lovable dashboard via Google OAuth. Supabase issues a cryptographically signed JWT with a one-hour expiration[1].
2. **Quota Check:** When the user initiates a "New Run";, Lovable triggers the Supabase Edge Function to atomically verify the user's quota[1, 2].
3. **Transmission:** If the quota check passes, Lovable opens the Replit application in a new tab, passing the Supabase JWT as a URL parameter (`?token=...`)[1].
4. **Sanitization:** Upon loading, the Vue frontend hosted on Replit immediately captures the token and strips it from the browser's address bar using `history.replaceState` . This ensures the token is only visible in the URL for approximately 50 milliseconds, preventing it from lingering in browser history[1, 2]. *Best Practice: Passing tokens in URL parameters is generally discouraged for persistent storage. Here, it is used strictly for a one-time cross-tab handoff. The immediate use of `history.replaceState` is critical to prevent the token from leaking via browser history or referrer headers.*
5. **Execution:** The Replit frontend attaches the token as an `Authorization: Bearer` header on all subsequent Axios calls to the Flask backend. The Flask backend validates the token';s signature against Supabase. If valid, the simulation runs; if expired, forged, or missing, the server returns a `401 Unauthorized` error[1]. *Best Practice: All subsequent API calls must use the `Authorization: Bearer` header. This standard allows the backend to verify the token against Supabase's signing key without exposing it in the URL, ensuring secure cross-platform verification.*

5. Deployment and Live Demo Walkthrough

Deployment in this ecosystem is highly streamlined. The Lovable dashboard is deployed with a single "Publish" click, generating a live URL. Concurrently, the Stripe webhook URL is registered to point to the Supabase instance, ensuring that successful checkouts immediately update the user's subscription status[1].

User Journey Walkthrough

In a live production environment, the user experiences a frictionless SaaS flow[1]:

- **Onboarding:** The user authenticates via Google OAuth and is assigned to the Free tier (0/3 runs).

- **Execution:** The user initiates a run. The network tab reveals the JWT being sent to the Edge Function. The Replit tab opens, the token is briefly visible in the URL, stripped, and the simulation (multi-agent dual universe) executes successfully.
- **Monetization:** Upon exhausting the three free runs, the UI presents an upgrade modal. The user is routed through Stripe Checkout. Upon completion, the webhook updates the database, and the user's dashboard reflects the "Pro" badge.
- **Security Verification:** Attempting to access the Replit URL directly via an incognito window without a valid token results in a strict `401 Unauthorized` response, proving the backend is successfully locked down[1].

6. SaaS Starter Kit Contents

To facilitate the rapid replication of this architecture, a comprehensive SaaS Starter Kit has been compiled. As advertised, this free resource allows developers to "Clone the repo. Connect Supabase. Deploy. No guesswork."[2] The kit comprises the following six components[1, 2]:

Component	Description
1. Lovable Template	An exported Lovable project containing the pre-built UI: landing page, login, dashboard, pricing, upgrade modal, and navbar. Includes Auth, Pricing, and Dashboard UI[1, 2].
2. Supabase Schema	SQL definitions for the `profiles`, `subscriptions`, and `simulation_runs` tables, complete with Row-Level Security (RLS) policies and the atomic quota Edge Function with row locking[1, 2].
3. Replit JWT Middleware	Python/Flask middleware for backend JWT verification on `/api/*` routes, CORS configuration for Lovable domains, and the Vue frontend token handler snippet (`history.replaceState`)[1, 2].
4. Stripe Config	Configuration details for setting up the webhook endpoint and the logic required to write subscription updates to the database[1].
5. Setup Guide	A step-by-step manual covering Lovable Cloud setup, Replit secret management, Stripe webhook configuration, and deployment instructions to go live[1, 2].

6. Architecture Diagram	A visual representation (PNG) illustrating the secure data flow and JWT handoff between Lovable, Supabase, and Replit[1].

7. Cost Estimates and Profitability

A critical aspect of transitioning from an open-source project to a SaaS is understanding unit economics. The following table outlines the estimated costs and revenue per user per month, factoring in LLM API costs (OpenRouter), database operations (Supabase), and payment processing fees (Stripe)[1].

User Type	OpenRouter Cost	Supabase Cost	Stripe Fee	Revenue
Free user (3 runs/month)	~$0.75–1.50	$0	$0	$0
Pro user (unlimited runs)	~$5–10	~$0.25	~$0.85	~$8–13 profit on $19

Note: The Pro tier assumes a reasonable "unlimited" usage pattern. The architecture's low overhead ensures a healthy profit margin of approximately $8 to $13 per paying user[1].

Modern AI Billing Strategies

While the Starter Kit implements a Tiered model (Free vs. Pro), modern AI SaaS products often benefit from a hybrid approach. A **Hybrid Subscription + Credit Top-up** model combines the recurring revenue of subscriptions with the flexibility of usage-based billing. This accommodates power users who exceed standard limits without forcing them into expensive enterprise tiers, while maintaining predictable revenue for the business.

8. Production Readiness Checklist

Before launching your AI SaaS, ensure the following operational and security best practices are met:

- **RLS Audits:** Verify that Row-Level Security policies on `profiles`, `subscriptions`, and `simulation_runs` strictly enforce data isolation. Users should never be able to query another user's runs.
- **Edge Function Timeouts:** AI operations can be unpredictable. Ensure your Supabase Edge Functions have appropriate timeout configurations and handle long-running tasks asynchronously if necessary to prevent user-facing errors.
- **Stripe Webhook Idempotency:** Implement idempotency keys in your webhook handler. Stripe may send the same event multiple times. Your logic must detect duplicate events to prevent double-charging or double-crediting users.

References

1. Video Script: "How I Turned an Open Source AI Repo Into a SaaS in One Evening — Supabase + Lovable + Stripe". Provided via user query context.
2. Knowledge Base File: `Screenshot6227.png`. Advertisement and component breakdown for the "SaaS Starter Kit".